c++ algorithm 详解
一、max()最大值、min()最小值、abs()求绝对值
二、swap()：交换函数
三、reverse()：翻转函数
四、sort()：排序函数
五、find():查找函数
六、upper_bound()、lower_bound()：查找函数
七、fill()：填充函数
八、count()：查找某值出现的次数
九、__gcd()：求最大公因数

c++ algorithm 详解

algorithm头文件介绍：

algorithm头文件是c++的标准算法库，它**主要应用在容器上。**因为所有的算法都是通过迭代器进行操作的，所以算法的运算实际上是和具体的数据结构相分离的，也就是说，具有低耦合性。因此，**任何数据结构都能使用这套算法库，只要它具有相应的迭代器类型**

一、max()最大值、min()最小值、abs()求绝对值

- `max()`：求**两个数**最大值
- `min()`：求**两个数**最小值
- `abs()`：求一个数的绝对值

代码：

```
#include<iostream>
#include<algorithm>
using namespace std;
int main() {
    int a = 3, b = 4;
    //求最大值
    int Max = max(a,b);
    //求最小值
    int Min = min(a,b);
    //求绝对值
    int Abs = abs(-3);
    cout<<"max="<<Max<<endl;
    cout<<"min="<<Min<<endl;
    cout<<"|Abs|="<<Abs<<endl;
    return 0;
 }
```

输出：

```
max=4
min=3
|Abs|=3
```

注意：

机器人工作室

1. `max()`和`min()`函数中的**参数只能是两个**，如果相求3个数的最大值，嵌套一下即可：

```
int Max=max(max(a,b),c);
int Min=min(min(a,b),c);
```

2. 写了`algorithm`头文件后，`max`就变成了函数名，**在自己定义变量时，要避免使用`max`，`min`等**
3. `max()`、`min()`也可以用来求字符的最大、最小值
4. `abs()`函数只能用于求整形变量的绝对值，而`#include<cmath>`中的`fabs()`函数还可用于求浮点数型变量的绝对值

二、swap()：交换函数

```
void swap(a,b)
```

用来交换a和b的值

代码：

```
#include<iostream>
#include<algorithm>
using namespace std;
int main()
{
    char a = 'A', b = 'B';
    swap(a,b);
    cout << "a="<<a <<endl<<"b="<< b;
    return 0;
}
```

输出：

```
a=B
b=A
```

三、reverse()：翻转函数

```
void reverse(BidirectionalIterator first,BidirectionalIterator last)
```

将容器[first，last)范围内的元素颠倒顺序放置

- 两个参数需要用首地址和尾地址来指定反转的区域

代码：

反转数组

机器人工作室

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int a[10]={0,1,2,3,4,5,6,7,8,9};
    reverse(a,a+10);
    for(int i=0;i<10;i++)
        cout<<a[i]<<" ";
    return 0;
}
```

输出：

```
9 8 7 6 5 4 3 2 1 0
```

反转字符串

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    string a="abcdefg";
    reverse(a.begin(),a.end());
    cout<<a;
    return 0;
}
```

输出：

```
gfedcba
```

机器人工作室

注意：

如果想在翻转时指定位数，则其为左闭右开区间，如reverse(a+2,a+4)，翻转下标为[2，4)之间的数

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int a[10]={0,1,2,3,4,5,6,7,8,9};
    reverse(a+2,a+4);
    for(int i=0;i<10;i++)
        cout<<a[i]<<" ";
    return 0;
}
```

输出：

```
    0 1 3 2 4 5 6 7 8 9
//下标 0 1 2 3 4 5 6 7 8 9
```

四、sort()：排序函数

```
void sort (RandomAccessIterator first, RandomAccessIterator last, Compare comp);
```

对x-y区间的数组、容器进行排序。**默认升序排列。**

详见sout()排序函数（c++）

五、find():查找函数

查找某数组指定区间x-y内是否有x，若有，则返回该位置的地址，若没有，则返回该数组第n+1个值的**地址。**需要转换!

代码：

对数组查找：

数组中查找是否有某值，需要满足这两个条件：

- p-a!=数组的长度。p是查找数值的地址，a是a[0]的地址。
- *p==x，也就是该地址指向的值等于我们要查找的值

最后输出p-a+1。p-a相当于x所在位置的地址-a[0]所在位置的地址，但因为是从0开始算，所以最后需要+1

```
#include<iostream>
#include<algorithm>
using namespace std;
int main()
{
    int a[5] = {1,2,3,4,5};

    int *p = find(a,a+5,3);              //定义指针，指向查找完成后返回的地址，5为a2数组长度
    if(((p-a) != 5) && (*p == 3))//如果指针p与数组a的起始地址相减不等于5（即p不等于a+5），并且指针p指向的值等于33，则说明查找成功。
        cout << (p-a+1);                        //输出所在位置
    return 0;
}
```

六、upper_bound()、lower_bound()：查找函数

- `upper_bound()`：查找第一个大于x的值的位置
- `lower_bound()`：查找第一个大于等于x的值的位置

同样是返回地址，用法和`find()`函数一样，限制条件也一样

七、fill()：填充函数

在区间内填充某一个值。同样适用于所有类型数组，容器。

机器人工作室

代码：

在数组中未赋值的地方填充0，相当于对数组初始化

```
#include<iostream>
#include<algorithm>
using namespace std;
int main()
{
    int a[10];
    fill(a,a+10,0);
    for(int i = 0; i < 10; i++)
        cout << a[i] << ' ';
    return 0;
}
```

输出：

```
0 0 0 0 0 0 0 0 0 0
```

八、count()：查找某值出现的次数

查找x 在某区间出现的次数

代码：

在数组中查找：

```
#include<iostream>
#include<algorithm>
using namespace std;
int main()
{
    int a[5] = {1,2,3,4,4};

    cout << count(a, a+5, 4);

    return 0;
 }
```

输出：

```
2
```

在字符串中查找：

机器人工作室

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    string a="2022";

    cout << count(a.begin(), a.end(), '2');

    return 0;
 }
```

输出

```
3
```

在容器中查找同理，只是需要用`iterator`迭代器或`begin()`、`end()`函数。

九、__gcd()：求最大公因数

- __gcd()用于求最大公因数
- 最小公倍数=两数乘积/最大公约数

```cpp
#include<iostream>
#include<algorithm>
using namespace std;
int main() {
    int a = 18, b = 32;
    int Gcd = __gcd(a,b);
    cout << Gcd;
    return 0;
 }
```

输出：

```
2
```

机器人工作室